# Celestra cheatsheet – v5.6.4 – https://github.com/Serrin/Celestra/

The `celestra` and/or the `CEL` objects contain these functions, except the polyfills. Example: `CEL.qsa("p");`

## Core API

```
javaHash(<data>[,hexa=false]);
b64Encode(<string>); b64Decode(<string>);
extend([deep,]<target>,<source1>[,srcN]);
sizeIn(<object>);
popIn(<obj>,<prop>);, forIn(<obj>,<cb>);
filterIn(<object>,<callback>);
delay + sleep(<ms>).then(<callback>);
inherit(<subclass>,<superclass>);
randomBoolean();
randomUUIDv7();
timestampID([size=21[,alphabet=BASE58]]);
nanoid([size=21[,alphabet="A-Za-z0-9-
_"]]);
BASE16; BASE32; BASE36; BASE58; BASE62;
WORDSAFEALPHABET;
getUrlVars([str=location.search]);
obj2string(<object>);
classof(<variable>[,type[,throw=false]]);
bind(<fn>,<context>); and unBind(<fn>);
constant(<value>); and identity(<value>);
noop(); and T(); and F();
assertEq(<msg>,<v1>,<v2>[,strict=true]);
assertNotEq(<m>,<v1>,<v2>[,strict=true]);
assertTrue(<message>,<value>);
assertFalse(<message>,<value>);
noConflict(); and VERSION;
```

## String API

```
strPropercase(<str>); strTitlecase(<s>);
strCapitalize(<string>);
strUpFirst(<str>); + strDownFirst(<str>);
strReverse(<string>);
strCodePoints(<string>);
strFromCodePoints(<collection>);
strAt(<string>,<index>[,newChar]);
strSplice(<str>,<index>,<count>[,add]);
strHTMLRemoveTags(<string>);
strHTMLEscape(<string>);
strHTMLUnEscape(<string>);
```

## DOM API

```
qsa(<selector>[,context]).forEach(<cb>);
qs(<selector>[,context]);
domReady(<callback>);
domCreate(<type>[,properties[,innerHTML]]);
domCreate(<element descriptive object>);
domToElement(<htmlString>);
domGetCSS(<element>[,property]);
domSetCSS(<element>,<property>,<value>);
domSetCSS(<element>,<properties>);
domFadeIn(<element>[,duration[,display]]);
domFadeOut(<element>[,duration]);
domFadeToggle(<elem.>[,duration[,display]]);
domShow(<element>[,display]);
domHide(<element>);
domToggle(<element>[,display]);
domIsHidden(<element>);
domScrollToTop();
domScrollToBottom();
domScrollToElement(<element>[,top=true]);
domSiblings(<element>);
domSiblingsPrev(<element>);
domSiblingsLeft(<element>);
domSiblingsNext(<element>);
domSiblingsRight(<element>);
domGetCSSVar(<name>);
domSetCSSVar(<name>,<value>);

importScript(<script1>[,scriptN]);
importStyle(<style1>[,styleN]);

setFullscreenOn(<selector> or <element>);
setFullscreenOff();
getFullscreen();

form2array(<form>);
form2string(<form>);
getDoNotTrack();
getLocation(<success>[,error]);
createFile(<filename>,<content>[,dType]);
```

## Type checking API

```
isMap(<value>); and isWeakMap(<v>);
isSet(<value>); and isWeakSet(<v>);
isNumber(<v>);  and isNumeric(<v>);
isFloat(<val>); and isBigInt(<v>);
isString(<v>);  and isChar(<val>);
isDate(<val>);  and isError(<val>);
isRegexp(<v>);  and isSymbol(<v>);
isElement(<v>); and isObject(<v>);
isDataView(<value>);
isBoolean(<value>);
isNull(<value>);
isUndefined(<value>);
isNullOrUndefined(<value>);
isNil(<value>);
isPlainObject(<value>);
isTruthy(<value>);
isFalsy(<value>);
isFunction(<v>); + isCallable(<v>);
isConstructorFn(<value>);
isGeneratorFn(<value>);
isAsyncGeneratorFn(<value>);
isAsyncFn(<value>);
isArraylike(<value>);
isTypedArray(<value>);
isArrayBuffer(<value>);
isPrimitive(<value>);
isPromise(<value>);
isIterator(<value>);
isIterable(<value>);
isEmptyObject(<value>);
isEmptyArray(<value>);
isEmptyMap(<value>);
isEmptySet(<value>);
isEmptyIterator(<value>);
isSameObject(<object1>,<object2>);
isSameArray(<array1>,<array2>);
isSameMap(<map1>,<map2>);
isSameSet(<set1>,<set2>);
isSameIterator(<iter1>,<iter2>);
```

| Collections API | | Polyfills |
|---|---|---|
| arrayCreate([length=0]);<br><br>arrayDeepClone(<array>);<br><br>arrayMerge(<target>,<source1>[,sourceN]);<br><br>arrayUnique(<collection>);<br><br>arrayAdd(<array>,<value>);<br>arrayClear(<array>);<br><br>arrayRemove(<array>,<value>[,all=false]);<br>arrayRemoveBy(<array>,<callback>[,all=false]);<br><br>arrayRange([start=0[,end=99[,step=1]]]);<br>iterRange([start=0[,step=1[,end=Infinity]]]);<br><br>arrayCycle(<collection>[,n=100]);<br>iterCycle(<iter>[,n=Infinity]);<br><br>arrayRepeat(<value>[,n=100]);<br>iterRepeat(<value>[,n=Infinity]);<br><br>slice(<collection>[,begin=0[,end=Infinity]]);<br>withOut(<collection>,<filterCollection>);<br><br>reduce(<collection>,<callback>[,initialvalue]);<br>count(<collection>,<callback>);<br><br>take(<collection>[,n=1]);<br>takeWhile(<collection>,<callback>);<br>takeRight(<collection>[,n=1]);<br>takeRightWhile(<collection>,<callback>);<br><br>drop(<collection>[,n=1]);<br>dropWhile(<collection>,<callback>);<br>dropRight(<collection>[,n=1]);<br>dropRightWhile(<collection>,<callback>); | forEach(<collection>,<callback>);<br>map(<collection>,<callback>);<br>enumerate(<collection>[,offset=0]);<br>entries(<collection>[,offset=0]);<br>size(<collection>);<br><br>every(<collection>,<callback>);<br>some(<collection>,<callback>);<br>none(<collection>,<callback>);<br><br>includes(<collection>,<value>);<br>contains(<collection>,<value>);<br>find(<collection>,<callback>);<br>findLast(<collection>,<callback>);<br><br>filter(<collection>,<callback>);<br>reject(<collection>,<callback>);<br>partition(<collection>,<callback>);<br><br>shuffle(<collection>);<br>min(<value1>[,valueN]);<br>max(<value1>[,valueN]);<br>sort(<collection>[,numbers=false]);<br>reverse(<collection>);<br><br>zip(<collection1>[,collectionN]);<br>unzip(<collection>);<br>zipObj(<collection1>,<collection2>);<br><br>item(<collection>,<index>);<br>nth(<collection>,<index>);<br>first(<collection>);<br>head(<collection>);<br>last(<collection>);<br>initial(<collection>);<br>tail(<collection>);<br>flat(<collection>);<br>concat(<collection1>[,collectionN]);<br>join(<collection>[,separator=","]); | Array.fromAsync();<br><br>Array.prototype.toReversed();<br><br>Array.prototype.toSorted();<br><br>Array.prototype.toSpliced();<br><br>Array.prototype.with();<br><br>crypto.randomUUID();<br><br>Error.isError();<br><br>globalThis;<br><br>Map.groupBy();<br><br>Object.groupBy();<br><br>Object.hasOwn();<br><br>TypedArray.prototype.toReversed();<br><br>TypedArray.prototype.toSorted();<br><br>TypedArray.prototype.with();<br><br>**Non-standard polyfills**<br><br>BigInt.prototype.toJSON();<br><br>window.AsyncFunction();<br><br>window.GeneratorFunction(); |

| Math API | | Abstract API |
|---|---|---|
| sum(<value1>[,valueN]);<br>avg(<value1>[,valueN]);<br>product(<value1>[,valueN]);<br><br>clamp(<value>,<min>,<max>);<br>minmax(<value>,<min>,<max>);<br><br>isEven(<value>);<br>isOdd(<value>);<br><br>randomInt([max]);<br>randomInt(<min>,<max>);<br><br>randomFloat([max]);<br>randomFloat(<min>,<max>);<br><br>inRange(<value>,<min>,<max>);<br><br>signbit(<value>); | toInt8(<value>);<br>toInt16(<value>);<br>toInt32(<value>);<br>toUInt8(<value>);<br>toUInt16(<value>);<br>toUInt32(<value>;<br>toBigInt64(<value>);<br>toBigUInt64(<value>);<br>toFloat16(<value>);<br>toFloat32(<value>);<br><br>isInt8(<value>);<br>isInt16(<value>);<br>isInt32(<value>);<br>isUInt8(<value>);<br>isUInt16(<value>);<br>isUInt32(<value>);<br>isBigInt64(<value>);<br>isBigUInt64(<value>);<br>isFloat16(<value>); | getIn(<object>,<property>);<br>getInV(<object>,<property>);<br>hasIn(<object>,<property>);<br>setIn(<object>,<property>,<value>);<br><br>toIndex(<value>);<br>toPropertyKey(<value>);<br>toInteger(<value>);<br>toArray(value);<br>toObject(<value>);<br><br>isIndex(<value>);<br>isPropertyKey(<value>);<br>isSameValue(<value1>,<value2>);<br>isSameValueZero(<value1>,<value2>);<br>isSameValueNonNumber(<value1>,<value2>);<br><br>type(<value>);<br>createDataProperty(<object>,<property>,<value>);<br>createMethodProperty(<object>,<property>,<value>); |

**Cookie API**

getCookie([name]);
hasCookie(<name>);
setCookie(<Options object>);
setCookie(<name>,<value>[,hours=8760[,path="/"[,domain[,secure[,SameSite="Lax"[,HttpOnly]]]]]]);
removeCookie(<Options object>);
removeCookie(<name>[,path="/"[,domain[,secure[,SameSite="Lax"[,HttpOnly]]]]]);
clearCookies(<Options object>);
clearCookies([path="/"[,domain[,sec[,SameSite="Lax"[,HttpOnly]]]]]);

**AJAX and CORS API**

getText(<url>,<success>);
getJson(<url>,<success>);

ajax(<Options object>);
**Options object properties (* = default value)**: url: *string*, data: *string*, queryType: **"ajax"/"cors"*, type: **"get"/"post"*, success: *function*, error: *function*, format: **"text"/"json"/"xml"*, user: *string*, password: *string*

| Removed Polyfills – Available in celestra-polyfills.dev.js and celestra-polyfills.min.js | | |
|---|---|---|
| **V3.1.0** | **v3.8.0** | **v5.6.0** |
| Array.from(); | | |
| Array.of(); | Array.prototype.values(); | Array.prototype.at(); |
| Array.prototype.copyWithin(); | | |
| Array.prototype.fill(); | Array.prototype.includes(); | Array.prototype.findLast(); |
| Array.prototype.find(); | | Array.prototype.findLastIndex(); |
| Array.prototype.findIndex(); | ChildNode.after(); | |
| Object.create(); | ChildNode.before(); | Array.prototype.flat(); |
| String.fromCodePoint(); | ChildNode.remove(); | Array.prototype.flatMap(); |
| String.prototype.codePointAt(); | ChildNode.replaceWith(); | |
| String.prototype.endsWith(); | | Array.prototype.group(); |
| String.prototype.startsWith(); | Element.prototype.closest(); | Array.prototype.groupToMap(); |
| | Element.prototype.getAttributeNames(); | |
| Math.acosh(); | Element.prototype.matches(); | Number.MIN_SAFE_INTEGER; |
| Math.asinh(); | Element.prototype.toggleAttribute(); | Number.MAX_SAFE_INTEGER; |
| Math.atanh(); | | |
| Math.cbrt(); | ParentNode.append(); | Object.fromEntries(); |
| Math.clz32(); | | |
| Math.cosh(); | ParentNode.prepend(); | Object.is(); |
| Math.expm1(); | | |
| Math.fround(); | String.prototype[Symbol.iterator]() | String.prototype.at(); |
| Math.hypot(); | String.prototype.includes(); | |
| Math.imul(); | String.prototype.repeat(); | String.prototype.matchAll(); |
| Math.log1p(); | | |
| Math.log10(); | NodeList.prototype.forEach(); | String.prototype.padStart(); |
| Math.log2(); | | String.prototype.padEnd(); |
| Math.sign(); | Object.assign(); | |
| Math.sinh(); | | String.prototype.replaceAll(); |
| Math.tanh(); | Object.entries(); | |
| Math.trunc(); | | String.prototype.trimStart(); |
| | Object.getOwnPropertyDescriptors(); | String.prototype.trimLeft(); |
| Number.EPSILON; | | String.prototype.trimEnd(); |
| Number.isNaN(); | Object.values(); | String.prototype.trimRight(); |
| isNaN(); | | |
| Number.isInteger(); | RegExp.prototype.flags; | Typedarray.prototype.at(); |
| Number.isFinite(); | | |
| Number.isSafeInteger(); | window.screenLeft; | TypedArray.prototype.findLast(); |
| Number.parseInt(); | window.screenTop; | TypedArray.prototype.findLastIndex(); |
| Number.parseFloat(); | | |